



Mestrado em Eng. Informática

2025/26

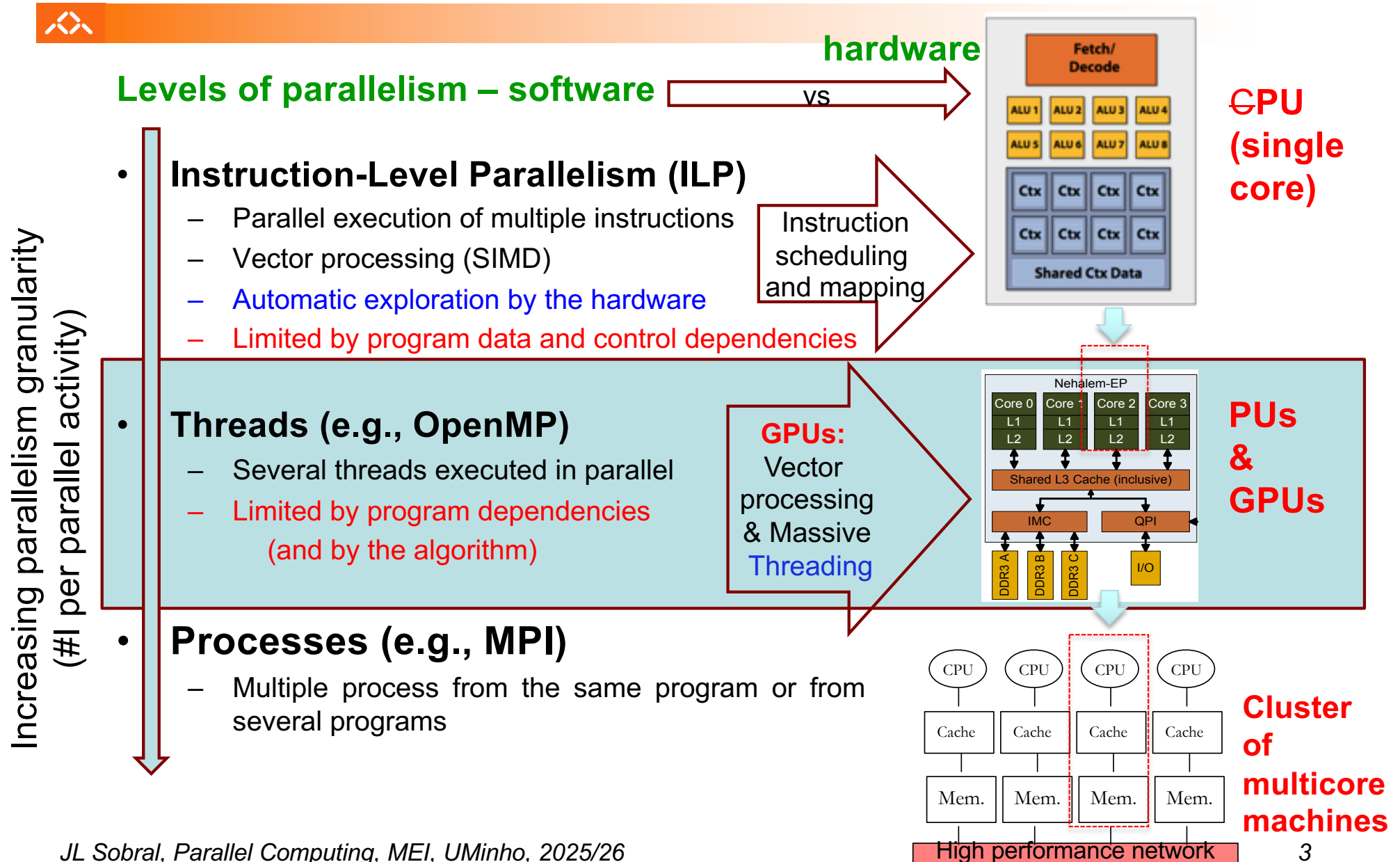
J.L. Sobral

Computing accelerators: GPUs



- 3ª sessão (16h-19h, ed1 2.21)
- Esta sessão não tem um objetivo específico
- Apoio genérico ao projeto:
 - utilização do cluster Deucalion (sbatch, etc);
 - utilização de ferramentas para medição de perfil de execução e exploração do código do projeto (Score-P/Cube GUI; perf, etc)
 - exploração de paralelismo com OpenMP e medição da escalabilidade.

Recap: Thread-level parallelism



Recap: GPU Arch. & vector proc.

- Similarities to **vector machines**:
 - Works well with data-level parallel problems
 - Scatter-gather transfers (note: to address lack of data locality)
 - Mask registers (note: to address control dependencies)
 - **Large register files**  **Hardware-based thread scheduling**
Registers are allocated to threads at creation
+ fast thread context switch (& lower cost)
- limits the maximum number of threads per Core
- Differences:
 - No scalar processor
 - Uses multithreading to hide [memory] latency

GK110 Kepler Architecture (k20 GPU on our cluster)

Kepler:
15 SMX
2880 CUDA-cores
October'13

Each **SMX** is
like a PU on a
multicore
processor
but SMXs
internally rely
on vector
processing
(see next slide)



GPU Organization (NVIDIA)

GPU Fine-grain multithreading
(1 clock cycle thread switch)

Warp scheduler

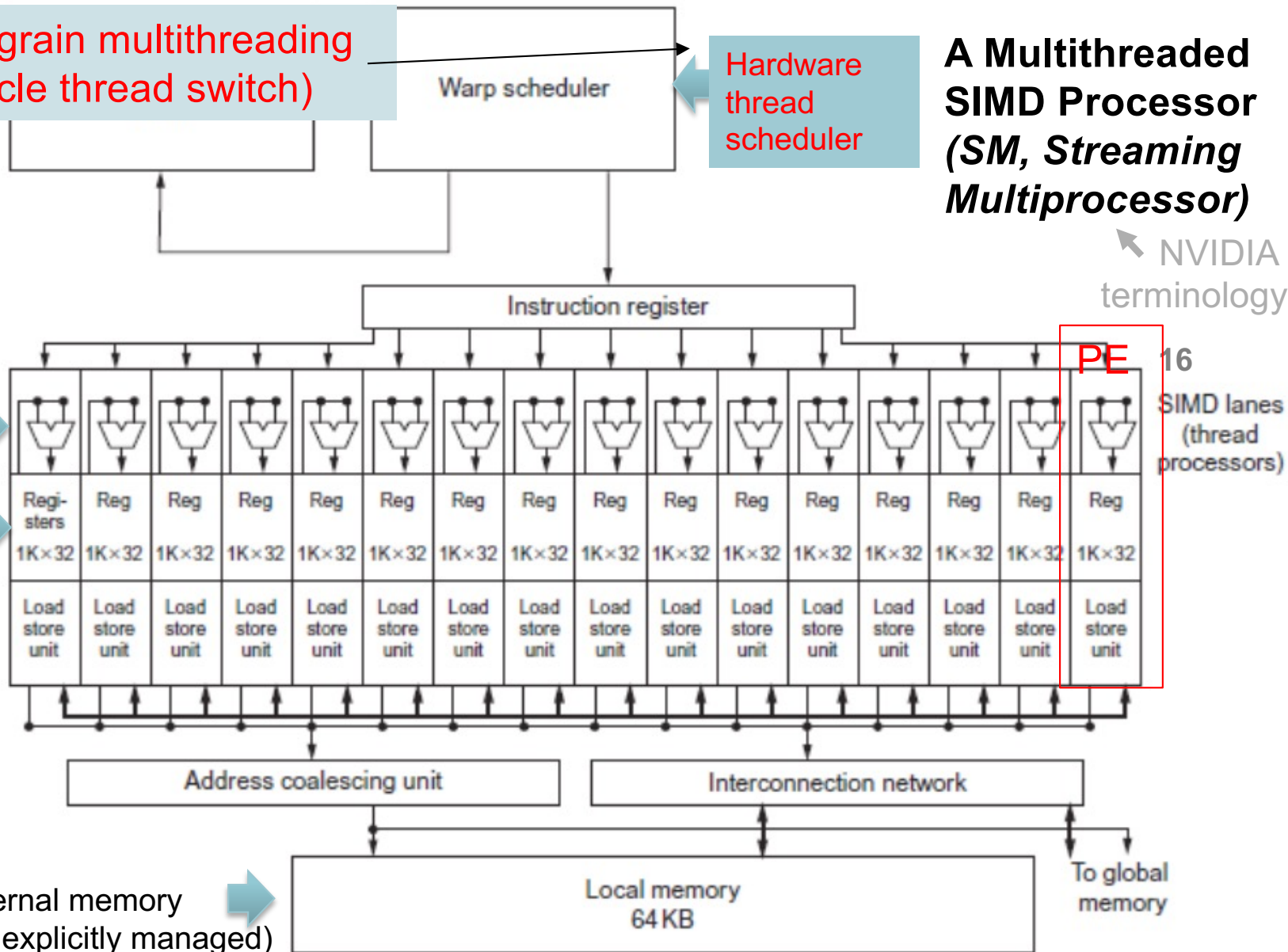
Hardware
thread
scheduler

A Multithreaded
SIMD Processor
(SM, Streaming
Multiprocessor)

NVIDIA
terminology

Vector
processing

Large
register
file
(1Kx32)



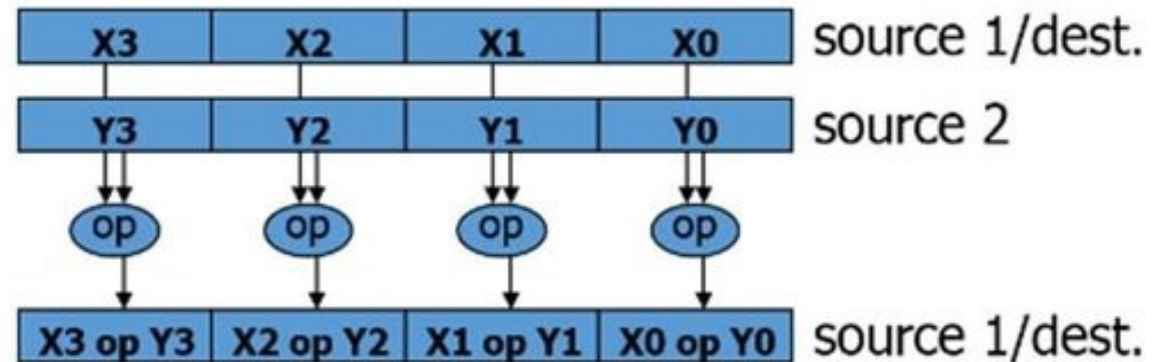
SIMD Data Parallelism: CPU vs. GPU

CPU

SIMD

1 instruction – multiple data

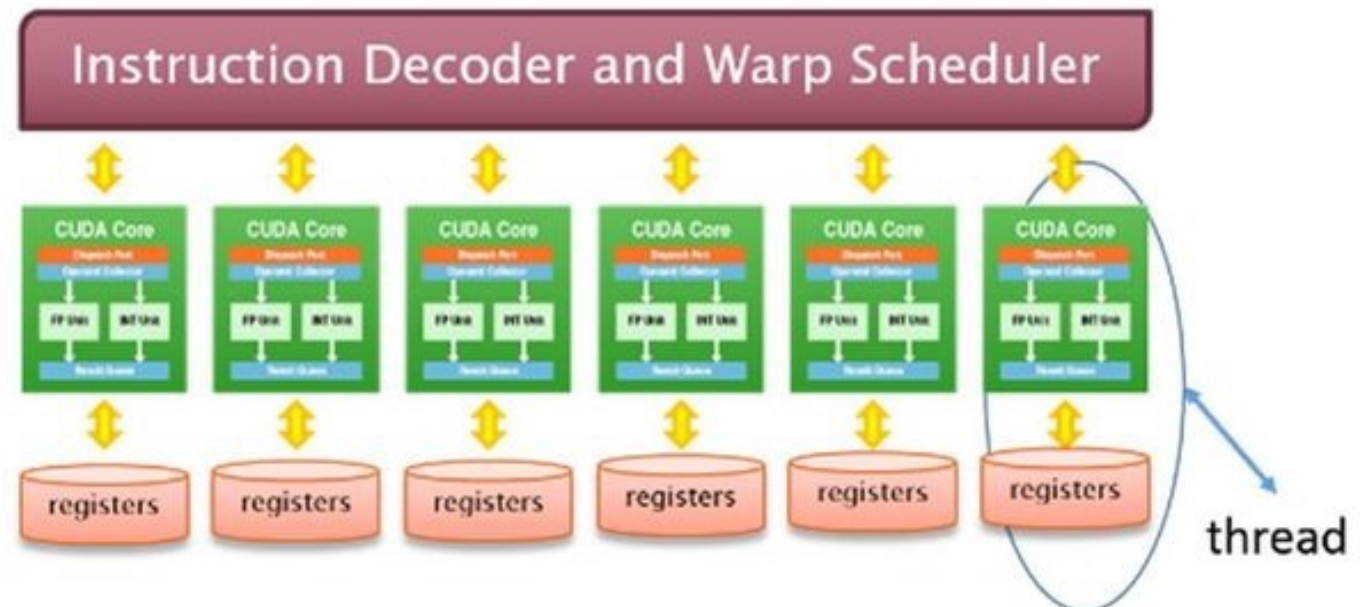
SSE2/3/4 – Neon – AltiVec
AVX – AVX2...



GPU

SIMT

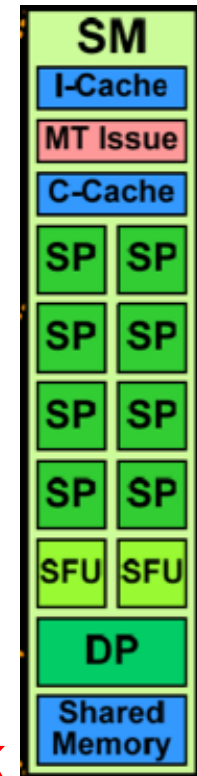
1 instruction – multiple threads



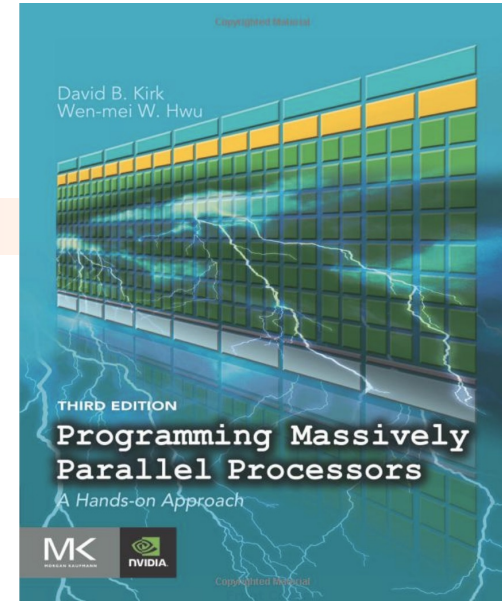
NVIDIA GPU Memory Structures

- Each SIMD Lane (**PE**) has private section of **off-chip DRAM**
 - “Private memory” (*Local Memory*)
 - Contains stack frame, spilling registers, and private variables
- Each multithreaded SIMD processor (**SM**) also has local memory (*Shared Memory*)
 - Shared by SIMD lanes / **threads within a block**
- Memory shared by SIMD processors (**SM**) is GPU Memory, **off-chip DRAM** (*Global Memory*)
 - Host can read and write GPU memory

NVIDIA terminology



The CUDA programming model



- **Compute Unified Device Architecture**
- CUDA is a programming model for
 - a multicore **CPU host** coupled to a many-core **device**, where
 - *devices* have wide SIMD/SIMT parallelism, and
 - the *host* and the *device* do not share memory
- CUDA provides:
 - a thread abstraction to deal with SIMD
 - synchr. & data sharing between small groups of threads
- CUDA programs are written in C with extensions
- OpenCL inspired by CUDA, but hw & sw vendor neutral
 - programming model essentially identical

CUDA Devices and Threads

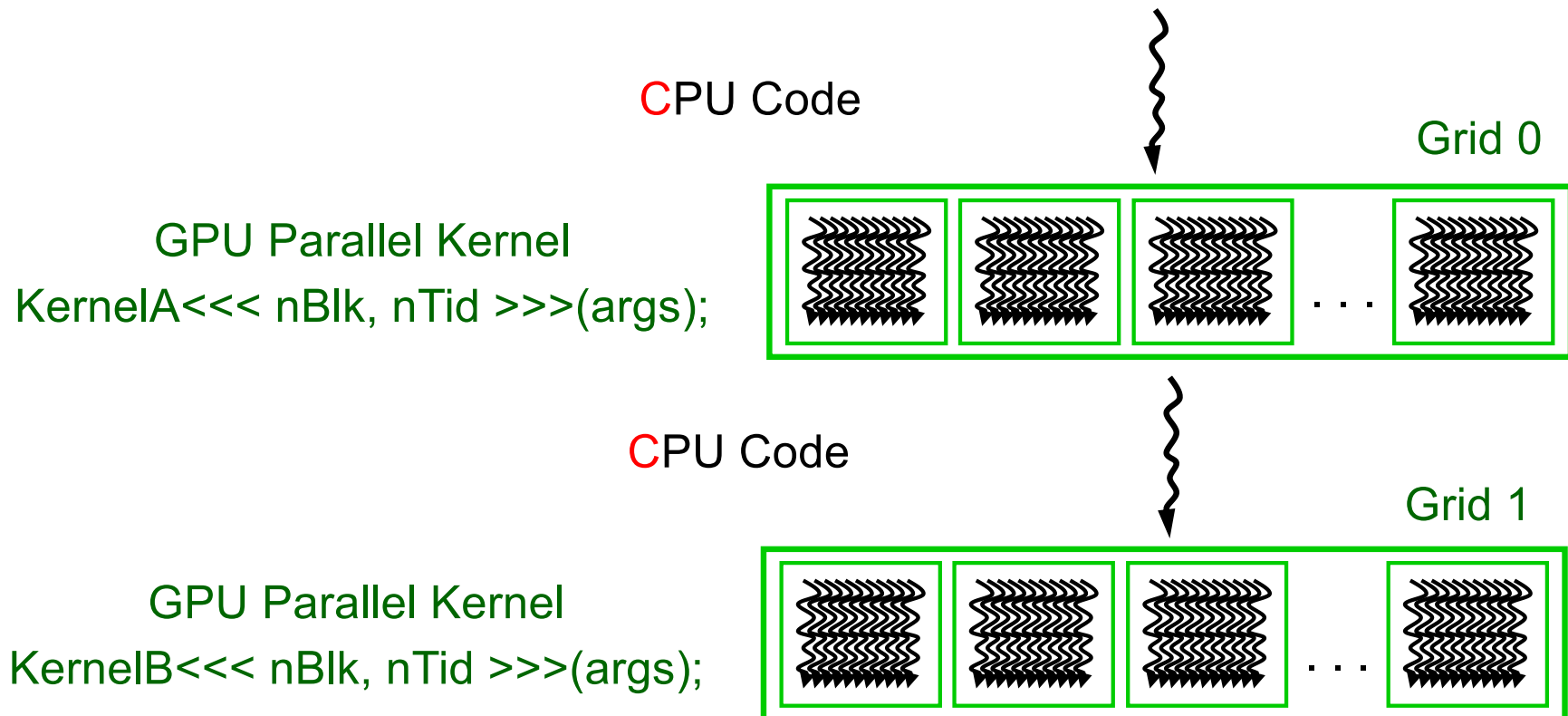


- A compute **device**
 - is a coprocessor to the **C**PU or **host**
 - has its own DRAM (**device memory**)
 - runs many **threads in parallel**
 - is typically a **GPU** but can also be another type of parallel processing device
- Data-parallel portions of an application are expressed as device **kernels** which run on many threads - **SIMT**
- Differences between GPU and **C**PU threads
 - GPU threads are extremely lightweight
 - very little overhead, **requires LARGE register file**
 - GPU needs 1000s of threads for full efficiency
 - multi-core **C**PU needs only a few

CUDA basic model: Single-Program Multiple-Data (SPMD)



- CUDA integrated **C**PU + GPU application C program
 - Serial C code executes on **C**PU
 - Parallel **K**ernel C code executes on GPU **thread blocks**



Code example



C with CUDA Extensions: C with a few keywords

```
void saxpy_serial(int n, float a, float *x, float *y)
{
    for (int i = 0; i < n; ++i)
        y[i] = a*x[i] + y[i];
}
// Invoke serial SAXPY kernel
saxpy_serial(n, 2.0, x, y);
```

Standard C Code

```
__global__ void saxpy_parallel(int n, float a, float *x, float *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i < n) y[i] = a*x[i] + y[i];
}
// Invoke parallel SAXPY kernel with 256 threads/block
int nblocks = (n + 255) / 256;
saxpy_parallel<<<nblocks, 256>>>(n, 2.0, x, y);
```

Parallel C Code

NVIDIA Confidential

CUDA Thread Block



- Programmer declares (Thread) Block:
 - Block size 1 to **512** concurrent threads
 - Block shape 1D, 2D, or 3D
 - Block dimensions in threads
- All threads in a Block execute the same thread program
- Threads share data and synchronize while doing their share of the work
- Threads have **thread id** numbers within Block
- Thread program uses **thread id** to select work and address shared data

CUDA Thread Block

